csiro-biomass

3rd Place Solution

Rank	3
Team	embee
Author	Mayukh Bhattacharyya
Topic ID	670901
Version	Original

Thanks to CSIRO and Kaggle for hosting and others for sharing their ideas throughout!

I was honestly surprised to end up so high. I bet on heavy ensemble to safeguard against a shake-up but didn't expect it would be of any help in climbing the leaderboard. My submission was a 19 model ensemble across different model types and CV strategies.

1. Model Architecture

I trained two family of models

- Metadata Models
- Non Metadata Models

Metadata Models

Initially I trained models with `vit_huge_plus_patch16_dinov3` backbone where the models have an auxiliary head to predict all the metadata which then feeds into the final heads to predict biomass. The models were trained with an auxiliary metadata loss along with the main loss. Later on I realized, metadata isn't really making much of a difference so I dropped it.

Non Metadata Models

I picked up the popular MambaBlock based model that everyone was using and made it work on 2 backbones and then eventually 3 backbones. First model had `vit_huge_plus_patch16_dinov3` and `convnextv2_large` as they complement well and the second model added `vit_base_patch16_224.mae` to the mix. From my experience, choice of model made very little improvement to the overall performance as all my models got 0.64+ over a 4/5 fold ensemble. However, the Huge Plus version of DinoV3 always outperformed the Large version for me.

All the models were 2-stream (splitting image into left & right) based except for one model where I tried the whole image as input. It was almost similar in performance.

All the models had 3 heads. GDM and Total were inferred from the 3 predicted values.

During training, I trained the backbones with 1/10th of the normal learning rate so as to not totally destroy the pre-trained weights.

```
class BiomassModelV2(nn.Module):
    """DINOv3 + Mamba Fusion + Multi-Head Regression"""

    def __init__(self, model1_name: str,
                 model2_name: str,
                 pretrained: bool = True
                ):
        super().__init__()
        self.model1_name = model1_name
        self.model2_name = model2_name

        self.backbone1 = timm.create_model(
```

```
            model1_name, pretrained=pretrained, num_classes=0, global_pool=''
        )
        self.backbone2 = timm.create_model(
            model2_name, pretrained=pretrained, num_classes=0, global_pool=''
        )
        nf1 = self.backbone1.num_features
        nf2 = 256
        nf = nf1 + nf2

        self.fusion1 = nn.Sequential(
            LocalMambaBlock(nf1, kernel_size=5, dropout=0.1),
            LocalMambaBlock(nf1, kernel_size=5, dropout=0.1)
        )
        self.fusion2 = nn.Sequential(
            LocalMambaBlock(nf2, kernel_size=5, dropout=0.1),
            LocalMambaBlock(nf2, kernel_size=5, dropout=0.1)
        )
        self.pool = nn.AdaptiveAvgPool1d(1)

        self.head_green = nn.Sequential(
            nn.Linear(2*nf, nf // 2), nn.GELU(), nn.Dropout(0.2),
            nn.Linear(nf // 2, 1), nn.Softplus()
        )
        self.head_dead = nn.Sequential(
            nn.Linear(2*nf, nf // 2), nn.GELU(), nn.Dropout(0.2),
            nn.Linear(nf // 2, 1), nn.Softplus()
        )
        self.head_clover = nn.Sequential(
            nn.Linear(2*nf, nf // 2), nn.GELU(), nn.Dropout(0.2),
            nn.Linear(nf // 2, 1), nn.Softplus()
        )

    def forward(self, x):
        # x is a tuple (left, right)
        left, right = x

        x_l1 = self.fusion1(self.backbone1(left))
        x_l2 = self.fusion2(self.backbone2(left).flatten(2))
        x_r1 = self.fusion1(self.backbone1(right))
        x_r2 = self.fusion2(self.backbone2(right).flatten(2))

        x_l1 = self.pool(x_l1.transpose(1, 2)).flatten(1)
        x_l2 = self.pool(x_l2.transpose(1, 2)).flatten(1)
        x_r1 = self.pool(x_r1.transpose(1, 2)).flatten(1)
        x_r2 = self.pool(x_r2.transpose(1, 2)).flatten(1)

        x_cat = torch.cat([x_l1, x_l2, x_r1, x_r2], dim=1)

        green = self.head_green(x_cat)
        dead = self.head_dead(x_cat)
        clover = self.head_clover(x_cat)
        gdm = green + clover
        total = gdm + dead

        # Return as a single tensor (batch, 5)
        return torch.cat([green, dead, clover, gdm, total], dim=1)
```

Data

Data was obviously the biggest factor of this competition and from the very beginning my focus was to extract the maximum information out of the training data provided. I used all the major augmentations available. Special mention to random grayscale to ensure the models learn features, not just color.

I improvised two extra augmentations which I believe helped

- One is a simple MixUp with a low lambda of 0.2-0.3.
- In the other, I made 3 random vertical cuts on the image to divide it into 4 segments and I permute the order of the segments. This is a good augmentation as a jumbled up image still has the same amount of biomass depicted in it. Similar to this, I switched left and right images with 50% probability.

I applied the above two augmentations together, not one or the other.

CV

I tried 2 CV techniques, or rather I should say fold creation techniques - as after the first month, I gave up on CV scores and solely trusted LB scores. To avoid overfitting, I chose to always use all the models of all the folds when I train a certain architecture. None of the CV strategies actually felt robust - there were always 1-2 folds that did great and 1-2 folds that were bad on LB.

- First CV technique was a GroupKFold one with the dates as groups.
- Second CV technique was slightly complex. It is based on the embedding distance CV that was open sourced. I extended it to divide each validation set into 2 equal subsets - easy and hard (based on nearest neighbors). At this point, I expected the model weights giving a higher score on the hard subset to align more with LB, but the exact opposite was true. The easy subset and LB had consistently better alignment. My guess is that the hard samples in test are really hard and models can get a higher score by being excellent on easy samples and okayish on hard samples compared to good on both samples.

```
def create_robust_cv(
    df,
    embeddings_col,
    target_names,
    n_splits=5,
    n_clusters=50,
    seed=42,
    k=5,
    distance_metric="euclidean",
):
    df = df.copy().reset_index(drop=True)

    # 1. Embeddings
    X = np.vstack(df[embeddings_col].values).astype(np.float32)
    X /= np.linalg.norm(X, axis=1, keepdims=True) + 1e-8

    # 2. Visual clustering
    kmeans = KMeans(
        n_clusters=n_clusters,
        random_state=seed,
```

```
        n_init=10,
    )
    df["visual_cluster"] = kmeans.fit_predict(X)

    # 3. Stratification target
    weights = {
        "Dry_Green_g": 0.1,
        "Dry_Dead_g": 0.1,
        "Dry_Clover_g": 0.1,
        "GDM_g": 0.2,
        "Dry_Total_g": 0.5,
    }

    composite = sum(df[t] * weights.get(t, 0) for t in target_names)
    try:
        df["target_bins"] = pd.qcut(composite, q=10, labels=False, duplicates="drop")
    except ValueError:
        df["target_bins"] = pd.qcut(composite, q=5, labels=False, duplicates="drop")

    # 4. CV split
    sgkf = StratifiedGroupKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=seed,
    )
    df["fold"] = -1
    df["val_difficulty"] = None
    df["mean_knn_dist_to_train"] = np.nan

    for f, (train_idx, val_idx) in enumerate(
        sgkf.split(df, df["target_bins"], groups=df["visual_cluster"])
    ):
        X_train, X_val = X[train_idx], X[val_idx]

        nn = NearestNeighbors(n_neighbors=k, metric=distance_metric)
        nn.fit(X_train)
        knn_dist, _ = nn.kneighbors(X_val)
        mean_dist = knn_dist.mean(axis=1)
        df.loc[val_idx, "mean_knn_dist_to_train"] = mean_dist

        # 5. EASY / HARD split
        order = np.argsort(mean_dist)
        mid = len(order) // 2

        easy = val_idx[order[:mid]]
        hard = val_idx[order[mid:]]

        df.loc[easy, ["fold", "val_difficulty"]] = [f, "easy"]
        df.loc[hard, ["fold", "val_difficulty"]] = [f + n_splits, "hard"]
    return df
```

Inference

- TTA actually made LB performance worse, so I turned it off. With heavy augmentation and ensemble, I didn't need it in the end.
- I obviously did use the scaling that I showed in the notebook I shared.. I ended up extending a little bit more. For clover, it really seems the data distribution is very different in test set. For Dry Dead, I feel all the

models predict conservatively towards the mean when the actual values are more towards the fringes. The scaling gave me a decent boost (< 0.01) on the Public LB.

Since neither the CV or LB was reliable, my strategy from the beginning was to train multiple architectures and ensemble. I didn't make any drastic changes in my models or training, mostly slow gradual improvements - based on stability of CV scores and improvements in LB scores.

Most of my individual architectures were very similar in performance (~0.64+) over 5-folds. The last model set I trained - the 3 backbone non-metadata model was my highest scoring submission with 0.67.

On a final note, two things I tried making work but failed were synthetic dataset and test time training, especially the latter - since the test data was so different. It's kinda exciting to read how some top teams got it to work for them.